

Uncertainty-Aware Audio Spoofing Detection

Monte Carlo Dropout vs **Bayesian Neural Network** on ASVspoof 2019 LA

Same network, two approaches to uncertainty: which probabilities can you trust — and when does the model know it's wrong?

Goal of the project

Compare two approaches to uncertainty estimation, fairly



Monte Carlo Dropout, a pragmatic stochastic regularization method, and a Bayesian NN using variational inference approach with an explicit posterior, under fully controlled conditions using the same CNN backbone, dataset, and evaluation protocol.

Trustworthiness of the predicted probabilities



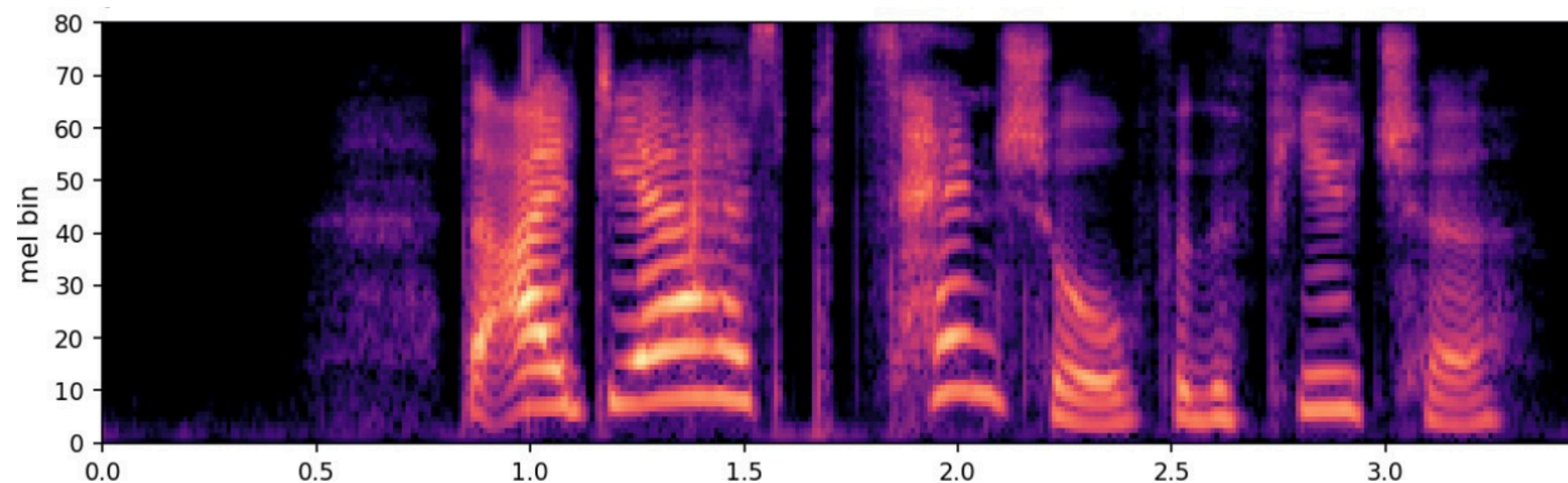
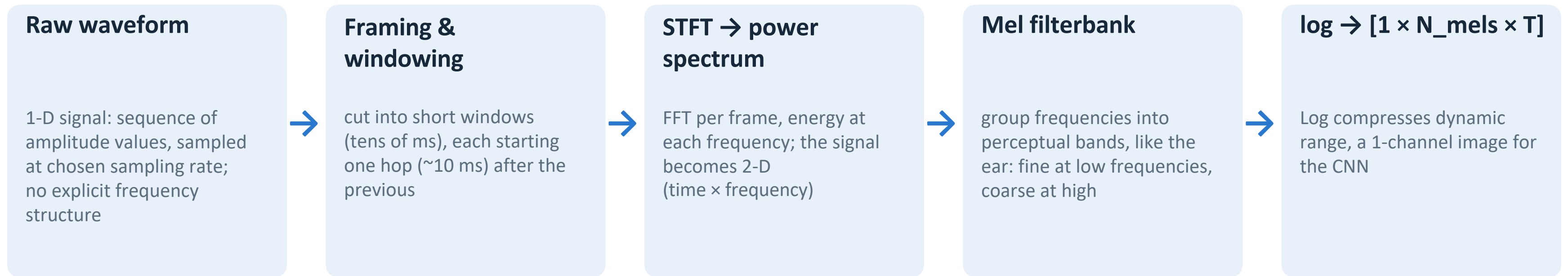
Aside from the standard EER metric, the project focuses on capturing the quality of what the models claim: a detector's outputs feed real decisions, so the predicted probabilities must be honest and confidence should reflect how often the model is actually right.

Uncertainty as a first-class output



Every prediction is accompanied by a measure of the model's own uncertainty. The project tests whether this signal is meaningful, so higher exactly where the model is more likely to be wrong.

From waveform to network input



This is what the CNN sees:

A $[1 \times 80 \times T]$ log-mel "image" of a real audio from the data. Time runs left to right, frequency bottom to top. The brighter a region, the more energy at that frequency and time; the dark regions are silence and pauses between words. Everything the network knows about the audio is contained in this single-channel image.

In this project: rather than reimplementing the pipeline above, we use the feature extractor of OpenAI's Whisper — a standard, extensively tested log-mel front end (16 kHz, 25 ms windows, 10 ms hop, 80 mel bins) already validated on hundreds of thousands of hours of speech. One adaptation: Whisper pads every input to 30 seconds, so spectrograms are trimmed back to the true utterance length before entering the network.

ASVspoof 2019 Dataset

split	used for	bonafide	spoof	total
train	model fitting	2,580	22,800	25,380
dev (subset)	validation, checkpoints	1,200	10,800	12,000
eval (full)	final metrics only	7,355	63,882	71,237

Class imbalance $\approx 1 : 9$

Handled with inverse-frequency class weights in the loss — $w = [8.84, 1.0]$ for [bonafide, spoof] — applied identically to both models. The weighting increases the contribution of the underrepresented bonafide class during optimization, reducing the tendency of the model to prioritize the majority spoof class.

Two classes, one task

Bonafide

genuine speech, recorded from real human speakers

Spoof

machine-generated speech: text-to-speech synthesis, or voice conversion applied to a real voice

The task: decide from the audio alone whether an utterance is genuine or machine-generated.

Approach 1 — Monte Carlo Dropout

Training objective — regularized cross-entropy

$$\mathcal{L}_{\text{MCD}} = \text{CrossEntropy}(y, \hat{y}) + \lambda \|\theta\|_2^2 \quad (\text{with class weights } \omega)$$

Training is completely standard. ω = inverse-frequency class weights, compensating the 1 : 9 class imbalance. The second term is L2 regularization, keeping the weights from growing too large.

The idea

At test time, dropout stays active and the input is passed through the network S times. Each pass randomly drops different feature maps, producing S plausible predictions: their average is the predictive distribution, and their disagreement quantifies the model's uncertainty — all without changing training.

The architecture

A ~3.8M-weight CNN — a convolutional stem, five residual blocks that convolve over the $[1 \times n_{\text{mels}} \times T]$ spectrogram and downsample only the time axis, statistics pooling (mean + std over time) to a fixed-size embedding, and a small fully-connected head producing the two class logits.

Where the noise is injected

Dropout2d ($p = 0.3$) after the first convolution of each residual block, plus standard dropout in the FC head.

Approach 2 — Stochastic Variational Inference

Training objective — negative ELBO

$$\mathcal{L}_{\text{ELBO}} = \text{CrossEntropy}(y, \hat{y}) + \beta \text{KL}(q(\mathbf{w}) \parallel p(\mathbf{w})), \quad \beta = \frac{1}{N}$$

Same data-fit term as Approach 1, the only difference is the KL penalty keeping the approximate posterior close to the $N(0, I)$ prior. Both q and p are Gaussian, so the KL has a closed form.

The idea

Instead of one fixed network, every conv / linear weight gets its own Gaussian, $q(w_i) = N(\mu_i, \sigma_i^2)$, learned by minimizing the negative ELBO. At test time the weights are sampled S times, giving S plausible predictions: their average is the prediction, their disagreement the uncertainty.

Where the randomness lives

Weights are sampled with the reparameterization trick, $w = \mu + \sigma \cdot \epsilon$, so gradients reach both μ and σ . Normalization layers and skip connections stay deterministic, and dropout is removed — the stochasticity comes only from the weights (7.5M stored parameters for the same 3.8M effective weights).

Training necessities

At initialization q is far from the prior and the KL is large enough to crush the data term and collapse q onto p before any fitting happens. Ramping β from 0 to $1/N$ over five epochs lets the network fit first and be regularized second. Weight decay is set to 0 — the KL against the $N(0, I)$ prior is already the regularizer, and adding L2 on top would penalize the weights twice.

Both models predict the same way

Monte Carlo prediction averaging, $S = 30$

Both approaches turn stochasticity into a prediction identically: run the input through the network S times and average the softmax outputs.

$$\hat{p}(y \mid x) = \frac{1}{S} \sum_{s=1}^S \text{softmax}\left(f(x; w^{(s)})\right), \quad S = 30.$$

Each pass uses a different **dropout mask (Approach 1)** or a different **weight sample (Approach 2)**. And since we keep all S predictions, their variability is itself a signal: how much they disagree becomes the uncertainty.

Same computation, different justification

For **Approach 2**, this is a Monte Carlo estimate of the posterior predictive — the predictions integrated over the approximate posterior $q(w)$:

$$p(y \mid x, \mathcal{D}) \approx \int p(y \mid x, w) q(w) dw$$

For **Approach 1** there is no posterior: the average is simply a pragmatic ensemble over stochastic passes. Whether that difference in justification shows up in the results is exactly what the evaluation tests.

Point-estimate baseline

Each model is also evaluated deterministically — dropout off for **Approach 1**, all weights at their means μ for **Approach 2** — quantifying what the averaging itself contributes.

Three lenses: EER, ECE, BALD

Equal Error rate

discrimination

“How well are the two classes separated?”

Equal Error Rate: sweep the decision threshold until the false-acceptance rate equals the false-rejection rate; report that common value. Threshold-free — it measures pure ranking ability of the score $p(\text{bonafide})$. The standard ASVspoof metric.

$$\text{EER} : \quad \text{FAR}(\tau^*) = \text{FRR}(\tau^*)$$

Expected Calibration Error

calibration

“When it says 90%, is it right 90% of the time?”

Expected Calibration Error: bin predictions into M equal-width confidence bins; ECE is the weighted average of $|\text{confidence} - \text{accuracy}|$ per bin. 0 = perfect. One number for the whole reliability diagram — but the absolute value hides whether the model is over- or underconfident.

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|$$

Bayesian Active Learning by Disagreement

epistemic uncertainty

“Do the sampled networks disagree?”

Entropy of the averaged prediction minus the average entropy of individual predictions. High only when samples are individually confident but mutually contradictory — isolating model uncertainty from shared noise.

$$\text{BALD}(x) = \mathcal{H}[\hat{p}(y | x)] - \frac{1}{S} \sum_{s=1}^S \mathcal{H}[p(y | x, w^{(s)})]$$

Model Results

3.99%

EER — MC Dropout wins discrimination
vs 7.94% for Bayesian NN

0.17

Negative Log Likelihood — Bayesian NN
wins probability quality
vs 0.41 for MC Dropout

model, prediction averaging, S = 30	EER %	ECE %	Mean BALD	NLL
MC Dropout	3.99	5.58	0.091	0.405
Bayesian NN	7.94	6.06	0.160	0.174

Same ECE, opposite behaviours

Calibration asks: when the model says it is 90% confident, is it right 90% of the time?

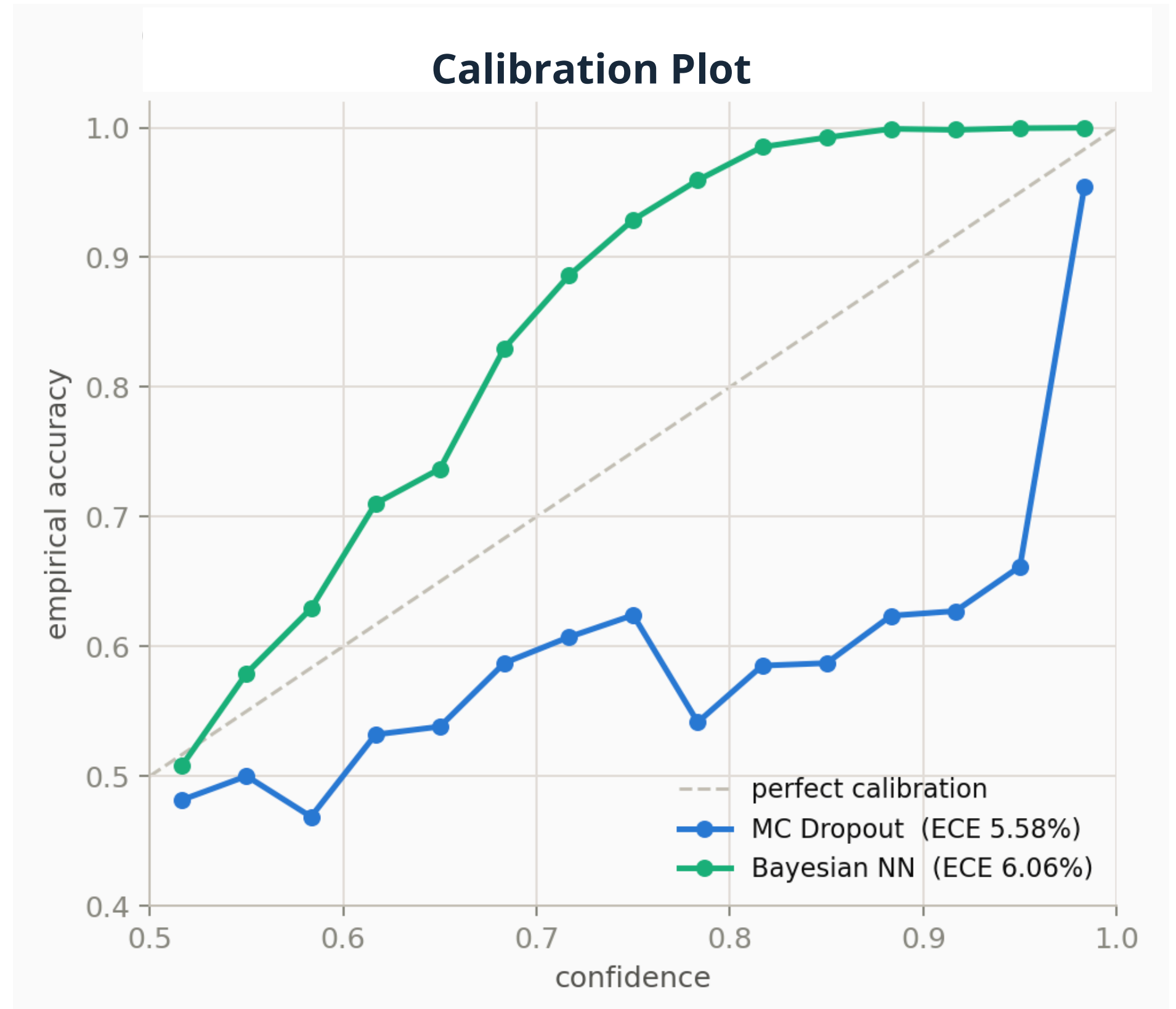
Nearly the same ECE — 5.58% against 6.06% — and opposite behaviour

MC Dropout - overconfident

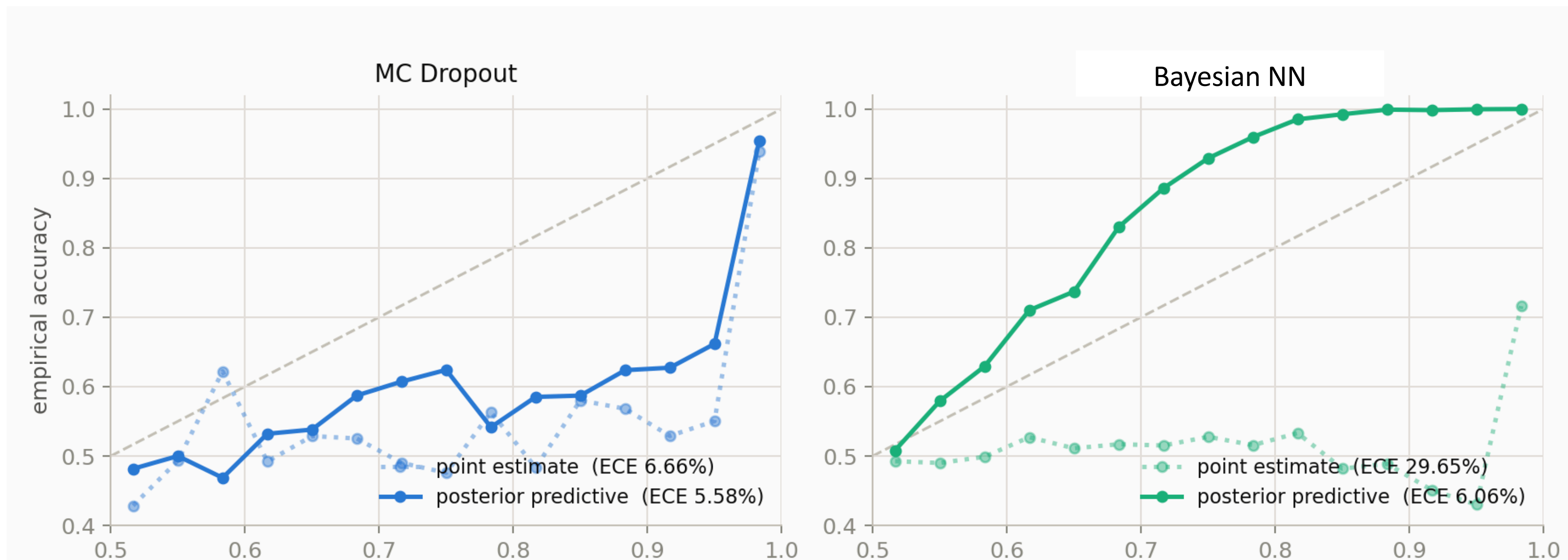
The whole curve sits below the diagonal. When it says 80% it is right about 58% of the time, and the gap never closes until the very last bin.

Bayes by Backprop — underconfident

The whole curve sits above the diagonal. When it says 70% it is right about 89% of the time, and above 0.9 it is essentially never wrong.



Marginalization is not optional



MC Dropout — dropout off → MC average

EER 4.09% → **3.99%**

ECE 6.66% → **5.58%**

The point estimate is already a sound classifier. Averaging barely moves EER, and improves the ECE a little.

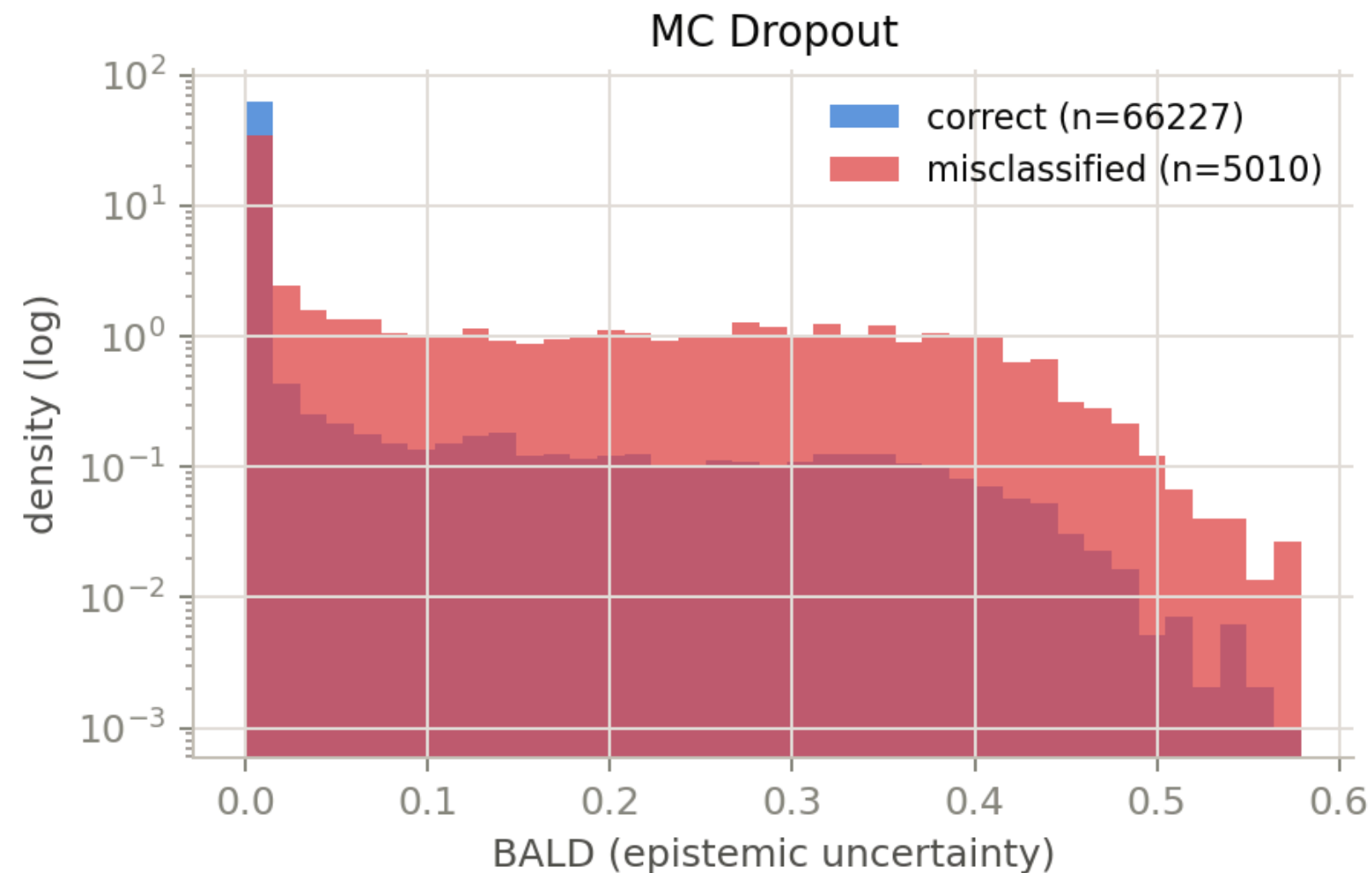
BayesianNN — posterior mean μ → MC average

EER 9.23% → **7.94%**

ECE 29.6% → **6.06%** *nearly 5× better*

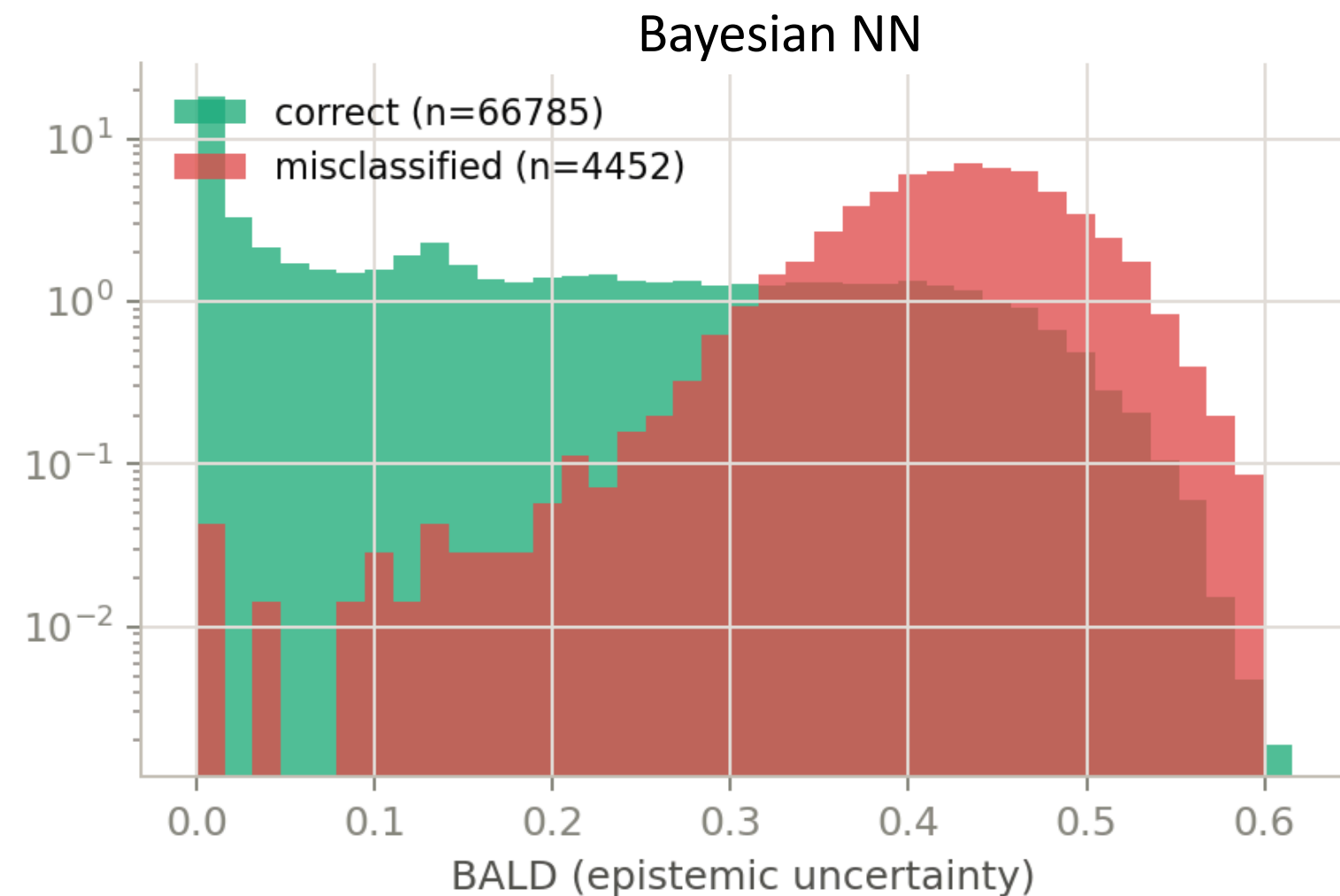
At its mean the model is a poor, wildly miscalibrated classifier. Only averaging over sampled weights turns the posterior into a usable classifier.

Only one model knows when it doesn't know



MC Dropout

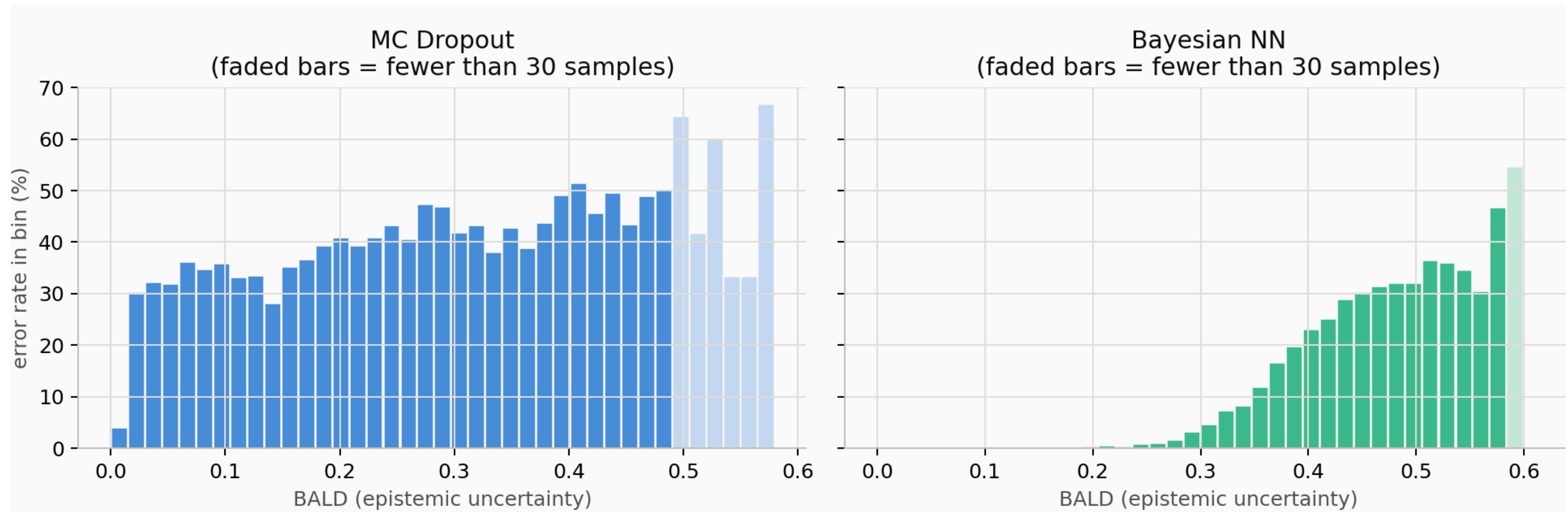
Errors at every level of uncertainty. Both histograms carry the same tall spike at $\text{BALD} \approx 0$. The red curve sits about ten times above the blue through the mid-range, but much of the error mass is inside that spike — where the model reports no disagreement at all.



Bayesian NN

It knows when it doesn't know. Below $\text{BALD} \approx 0.2$ there are almost no errors at all — in that range the model is essentially always right. As the uncertainty rises the mistakes start to appear, gathering into their own mode at 0.40–0.45. Where the model is unsure is where it is wrong.

Does high uncertainty mean high error?



MC Dropout

A switch, not a dial. Only the very first bin is safe: at BALD ≈ 0 the error rate is about 4%. Immediately above it the rate jumps to 30% and then drifts up to roughly 50%. Knowing that BALD is 0.1 rather than 0.4 tells you almost nothing.

Bayesian NN

The error rate follows the uncertainty. Below BALD ≈ 0.2 essentially nothing is wrong. Above it the rate climbs smoothly and monotonically to around 35–45%. Here the value of BALD, not merely its presence, says how far to trust the prediction.

Conclusions

Both models fail, but in opposite directions

MC Dropout is systematically overconfident, Bayesian NN systematically underconfident. Same scalar ECE, opposite behavior — and underconfidence is the safer failure mode for a security system

Marginalization is essential for the explicit posterior

Bayesian NN's point estimate is a broken classifier (9.2% EER, 29.6% ECE); MC Dropout's point estimate is nearly as good as its full average

The payoff

MC Dropout trains as an ordinary network. The Bayesian NN is harder to train — a μ and σ per weight, a KL warm-up, more tuning — and it still loses on EER. What you get in return is a model that separates its own mistakes: below a certain BALD it is almost always right, and the errors concentrate at high uncertainty. You know when to trust it.